# P3235R0
# std::print more types faster with less memory

Draft Proposal, 2024-04-13

**Author:**
Victor Zverovich

**Audience:**
LEWG

**Project:**
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

---

## Table of Contents

**1 Introduction**

**2 Proposal**

**3 Wording**

**References**
Informative References

"No work is less work than some work" - Andrei Alexandrescu

## § 1. Introduction

[P3107] enabled an efficient implementation of `std::print` and applied the optimization to fundamental and string types. The current paper applies this important optimization to the remaining standard types.

## § 2. Proposal

[P3107] "Permit an efficient implementation of std::print" brought significant speedups (from 20% in the original benchmarks to 2x in [SO-LARGE-DATA]) to `std::print` and eliminated the need for dynamic memory allocations in the common case by enabling direct writes into the stream buffer. To expedite the adoption of the fix, [P3107] limited the scope to fundamental and string types but it is, of course, beneficial to enable this optimization for other standard types that have formatters. This was discussed in LEWG that encouraged writing such paper (for ranges):

> LEWG requests for an additional paper to fix formatters for ranges

The current paper proposes opting in formatters for ranges and other standard types into this optimization.

Here is a list of standard formatters that are not yet opted into the `std::print` optimization.

Date and time formatters [time.syn]:

```
template<class Rep, class Period, class charT>
  struct formatter<chrono::duration<Rep, Period>, charT>;
template<class Duration, class charT>
  struct formatter<chrono::sys_time<Duration>, charT>;
template<class Duration, class charT>
  struct formatter<chrono::utc_time<Duration>, charT>;
template<class Duration, class charT>
  struct formatter<chrono::tai_time<Duration>, charT>;
template<class Duration, class charT>
  struct formatter<chrono::gps_time<Duration>, charT>;
template<class Duration, class charT>
  struct formatter<chrono::file_time<Duration>, charT>;
template<class Duration, class charT>
  struct formatter<chrono::local_time<Duration>, charT>;
template<class Duration, class charT>
  struct formatter<chrono::local-time-format-t<Duration>, charT>;
template<class charT> struct formatter<chrono::day, charT>;
template<class charT> struct formatter<chrono::month, charT>;
template<class charT> struct formatter<chrono::year, charT>;
template<class charT> struct formatter<chrono::weekday, charT>;
template<class charT> struct formatter<chrono::weekday_indexed, charT>;
template<class charT> struct formatter<chrono::weekday_last, charT>;
template<class charT> struct formatter<chrono::month_day, charT>;
template<class charT> struct formatter<chrono::month_day_last, charT>;
```

```
template<class charT> struct formatter<chrono::month_weekday, charT>;
template<class charT> struct formatter<chrono::month_weekday_last, charT>;
template<class charT> struct formatter<chrono::year_month, charT>;
template<class charT> struct formatter<chrono::year_month_day, charT>;
template<class charT> struct formatter<chrono::year_month_day_last, charT>;
template<class charT> struct formatter<chrono::year_month_weekday, charT>;
template<class charT> struct formatter<chrono::year_month_weekday_last, cha
template<class Rep, class Period, class charT>
  struct formatter<chrono::hh_mm_ss<duration<Rep, Period>>, charT>;
template<class charT> struct formatter<chrono::sys_info, charT>;
template<class charT> struct formatter<chrono::local_info, charT>;
template<class Duration, class TimeZonePtr, class charT>
  struct formatter<chrono::zoned_time<Duration, TimeZonePtr>, charT>;
```

`Rep` is an arithmetic type, `Period` is `std::ratio<...>`, `Duration` is `std::duration<...>` and `charT` is `char` or `wchar_t` so all chrono formatters except the one for `std::zoned_time` can be unconditionally opted into the optimization. The formatter for `std::zoned_time` can be opted in for the default `TimeZonePtr` (`const std::chrono::time_zone*`) but not arbitrary user-provided `TimeZonePtr` that can be potentially locking.

`std::thread::id` formatter [thread.thread.id]:

```
template<class charT> struct formatter<thread::id, charT>;
```

Stacktrace formatters [stacktrace.syn]:

```
// [stacktrace.format], formatting support
template<> struct formatter<stacktrace_entry>;
template<class Allocator> struct formatter<basic_stacktrace<Allocator>>;
```

`std::vector<bool>` formatter [vector.syn]:

```
// [vector.bool.fmt], formatter specialization for vector<bool>
template<class T, class charT> requires is-vector-bool-reference<T>
  struct formatter<T, charT>;
```

`std::filesystem::path` formatter added in [P2845] and, as of 14 Apr 2024, in the process of being merged into the standard draft:

```
// [fs.path.fmt], formatter
template<class charT> struct formatter<filesystem::path, charT>;
```

`std::thread::id`, stacktrace, `std::vector<bool>` and `std::filesystem::path` formatters don't invoke any user code and can be opted into the optimization.

Tuple formatter [format.tuple]:

```
template<class charT, formattable<charT>... Ts>
struct formatter<pair-or-tuple<Ts...>, charT> {
  ...
};
```

The tuple formatter can be opted in if all the element formatters are opted in.

Range formatter [format.syn]:

```
// [format.range.fmtmap], [format.range.fmtset], [format.range.fmtstr], spe
template<ranges::input_range R, class charT>
  requires (format_kind<R> != range_format::disabled) &&
           formattable<ranges::range_reference_t<R>, charT>
struct formatter<R, charT> : <i>range-default-formatter</i>&lt;format_kind<
```

`std::queue` and `std::priority_queue` formatters [queue.syn]:

```
// [container.adaptors.format], formatter specialization for queue
template<class charT, class T, formattable<charT> Container>
  struct formatter<queue<T, Container>, charT>;

...
```

```
// [container.adaptors.format], formatter specialization for priority_queue
template<class charT, class T, formattable<charT> Container, class Compare>
  struct formatter<priority_queue<T, Container, Compare>, charT>;
```

`std::stack` formatter [stack.syn]:

```
// [container.adaptors.format], formatter specialization for stack
template<class charT, class T, formattable<charT> Container>
  struct formatter<stack<T, Container>, charT>;
```

Range and container adaptor formatters are the most interesting case because formatting requires iterating and user-defined iterators can be locking, at least in principle. None of the standard containers, ranges and container adaptors and even common concurrent containers such as `concurrent_vector` from [TBB] provide locking iterators. For this reason, the current paper proposes opting range and adaptor formatters into the optimization by default. Other languages such as Java (see [P3107]) and even Rust don't try to prevent deadlocks when printing any user-defined types to a C stream and for iterators those are very unlikely. As shown in [P3107] examples of such deadlocks are pretty contrived and may indicate other issues (bugs) in the program such as incorrect lock scope.

And finally this paper proposes renaming `vprint_(non)unicode` and `vprint_(non)unicode_locking` to `vprint_(non)unicode_buffered` and `vprint_(non)unicode` respectively. The current naming is misleading because all of these functions are locking and "nonlocking" overloads confusingly call "locking" ones. In POSIX and other languages the default is locking so the new naming is more consistent with standard practice. The new naming reflects the fact that the main difference is buffering of all of the output.

## § 3. Wording

Modify [format.formatter.spec] as indicated:

...

The parse member functions of these formatters interpret the format specification as a *std-format-spec* as described in [format.string.std]. ~~In addition, for each type `T` for which a `formatter` specialization is provided above, each of the headers provides the following specialization:~~

Unless specified otherwise, for each type `T` for which a `formatter` specialization is provided by the library, each of the headers provides the following specialization:

```
template<> inline constexpr bool enable_nonlocking_formatter_optimization<T
```

...

Modify [time.format] as indicated:

...

If the *chrono-specs* is omitted, the chrono object is formatted as if by streaming it to `basic_ostringstream<charT> os` with the formatting locale imbued and copying `os.str()` through the output iterator of the context with additional padding and adjustments as specified by the format specifiers.

[*Example 3*:

```
string s = format("{:=>8}", 42ms);      // value of s is "====42ms"
```

— *end example*]

For `chrono::zoned_time` the library only provides the following specialization of `enable_nonlocking_formatter_optimization`:

```
template<class Duration>
  inline constexpr bool enable_nonlocking_formatter_optimization<
    chrono::zoned_time<Duration, const std::chrono::time_zone*>> = true;
```

```
template<class Duration, class charT>
  struct formatter<chrono::sys_time<Duration>, charT>;
```

...

Modify [format.tuple] as indicated:

For each of `pair` and `tuple`, the library provides the following formatter specialization where *pair-or-tuple* is the name of the template:

```
namespace std {
  template<class charT, formattable<charT>... Ts>
  struct formatter<pair-or-tuple<Ts...>, charT> {
    ...
  };

  template<class... Ts>
    inline constexpr bool enable_nonlocking_formatter_optimization<pair-or-
      (enable_nonlocking_formatter_optimization<Ts> && ...);
}
```

Modify [format.syn] as indicated:

```
...

// [format.range.fmtmap], [format.range.fmtset], [format.range.fmtstr], spe
template<ranges::input_range R, class charT>
  requires (format_kind<R> != range_format::disabled) &&
           formattable<ranges::range_reference_t<R>, charT>
struct formatter<R, charT> : range-default-formatter<format_kind<R>, R, cha

template<ranges::input_range R>
    requires (format_kind<R> != range_format::disabled)
  inline constexpr bool enable_nonlocking_formatter_optimization<R> =
    enable_nonlocking_formatter_optimization<remove_cvref_t<ranges::range_

// [format.arguments], arguments
// [format.arg], class template basic_format_arg
template<class Context> class basic_format_arg;

...
```

Modify [print.fun] as indicated:

```
template<class... Args>
  void print(FILE* stream, format_string<Args...> fmt, Args&&... args);
```

*Effects*: Let `locksafe` be `(enable_nonlocking_formatter_optimization<remove_cvref_t<Args>> && ...)`. If the ordinary literal encoding ([lex.charset]) is UTF-8, equivalent to:

```
locksafe
  ? vprint_unicode~~_locking~~(stream, fmt.str, make_format_args(args...))
  : vprint_unicode_buffered(stream, fmt.str, make_format_args(args...));
```

Otherwise, equivalent to:

```
locksafe
  ? vprint_nonunicode~~_locking~~(stream, fmt.str, make_format_args(args...))
  : vprint_nonunicode_buffered(stream, fmt.str, make_format_args(args...));
```

...

```
void vprint_unicode_buffered(FILE* stream, string_view fmt, format_args arg
```

*Effects*: Equivalent to:

```
string out = vformat(fmt, args);
vprint_unicode~~_locking~~(stream, "{}", make_format_args(out));
```

```
void vprint_unicode~~_locking~~(FILE* stream, string_view fmt, format_args args
```

...

```
void vprint_nonunicode_buffered(FILE* stream, string_view fmt, format_args
```

*Effects*: Equivalent to:

```
string out = vformat(fmt, args);
vprint_nonunicode_locking("{}", make_format_args(out));
```

```
void vprint_nonunicode_locking(FILE* stream, string_view fmt, format_args a
```

...

# § References

## § Informative References


**[P2845]**
Victor Zverovich. *Formatting of std::filesystem::path*. URL: https://wg21.link/p2845

**[P3107]**
Victor Zverovich. *Permit an efficient implementation of std::print*. URL: https://wg21.link/p3107

**[SO-LARGE-DATA]**
Matthew Busche. *How to use {fmt} with large data*. URL: https://stackoverflow.com/a/78457454/471164

**[TBB]**
*oneAPI Threading Building Blocks (oneTBB)*. URL: https://oneapi-src.github.io/oneTBB/